

Лабораторная работа №1. Лексематический анализ текста формата JSON

1. Цель и содержание

Цель лабораторной работы: Научиться проводить лексематический анализ текста в формате JSON, научиться выделять лексемы в тексте.

2. Теоретическая часть

Лексический анализатор (или сканер) – это часть компилятора, которая читает литеры программы на исходном языке и строит из них слова (лексемы) исходного языка. На вход лексического анализатора поступает текст исходной программы, а выходная информация передается для дальнейшей обработки компилятором на этапе синтаксического анализа и разбора.

С теоретической точки зрения лексический анализатор не является обязательной, необходимой частью компилятора. Его функции могут выполняться на этапе синтаксического разбора. Однако существует несколько причин, исходя из которых в состав практически всех компиляторов включают лексический анализ. Эти причины заключаются в следующем:

- упрощается работа с текстом исходной программы на этапе синтаксического разбора и сокращается объем обрабатываемой информации, так как лексический анализатор структурирует поступающий на вход исходный текст программы и выкидывает всю незначущую информацию;

- для выделения в тексте и разбора лексем возможно применять простую, эффективную и теоретически хорошо проработанную технику анализа, в то время как на этапе синтаксического анализа конструкций исходного языка используются достаточно сложные алгоритмы разбора;

- сканер отделяет сложный по конструкции синтаксический анализатор от работы непосредственно с текстом исходной программы, структура которого может варьироваться в зависимости от версии входного языка – при такой конструкции компилятора при переходе от одной версии языка к другой достаточно только перестроить относительно простой сканер.

Функции, выполняемые лексическим анализатором, и состав лексем, которые он выделяет в тексте исходной программы, могут меняться в зависимости от версии компилятора. В основном лексические анализаторы выполняют исключение из текста исходной программы комментариев и незначащих пробелов, а также выделение лексем следующих типов: идентификаторов, строковых, символьных и числовых констант, ключевых (служебных) слов входного языка.

В простейшем случае фазы лексического и синтаксического анализа могут выполняться компилятором последовательно. Но для многих языков программирования информации на этапе лексического анализа может быть недостаточно для однозначного определения типа и границ очередной лексемы. Иллюстрацией такого случая может служить пример оператора программы на языке Фортран, когда по части текста `DO 10 I=1...` невозможно определить тип оператора (а соответственно, и границы лексем). В случае `DO 10 I=1.15` – это будет присвоение вещественной переменной `DO10I` значения константы `1.15` (пробелы в Фортране игнорируются), а в случае `DO 10 I=1,15` – это цикл с перечислением от 1 до 15 по целочисленной переменной `I` до метки `10`.

В большинстве компиляторов лексический и синтаксический анализаторы – это взаимосвязанные части. Лексический разбор исходного текста в таком варианте выполняется поэтапно так, что синтаксический анализатор, выполнив разбор очередной конструкции языка, обращается к сканеру за следующей лексемой. При этом он может сообщить информацию о том, какую лексему следует ожидать. В процессе разбора может даже происходить «откат назад», чтобы выполнить анализ текста на другой основе. В дальнейшем будем исходить из предположения, что все лексеммы могут быть однозначно выделены сканером на этапе лексического разбора.

Вид представления информации после выполнения лексического анализа целиком зависит конструкции компилятора. Но в общем виде ее можно представить как таблицу лексем, которая в каждой строчке должна содержать информацию о виде лексемы, ее типе и, возможно, значении.

Обычно такая таблица имеет два столбца: первый – строка лексемы, второй – указатель на информацию о лексеме, может быть включен и третий столбец – тип лексем.

Лексический анализатор имеет дело с такими объектами, как различного рода константы и идентификаторы (к последним относятся и ключевые слова). Язык констант и идентификаторов в большинстве случаев является регулярным – то есть, может быть описан с помощью регулярных (праволинейных или леволинейных) грамматик. Распознавателями для регулярных языков являются конечные автоматы. Существуют правила, с помощью которых для любой регулярной грамматики может быть построен недетерминированный конечный автомат, распознающий цепочки языка, заданного этой грамматикой.

Недетерминированный конечный автомат задается пятеркой:

$$M=(Q, \Sigma, \delta, q_0, F),$$

где:

Q – конечное множество состояний автомата;

Σ – конечное множество допустимых входных символов;

δ – заданное отображение множества $Q^*\Sigma$ во множество подмножеств $P(Q)$ $\delta: Q^*\Sigma \rightarrow P(Q)$ (иногда δ называют функцией переходов автомата);

$q_0 \in Q$ – начальное состояние автомата;

$F \subseteq Q$ – множество заключительных состояний автомата.

Работа автомата выполняется по тактам. На каждом очередном такте i автомат, находясь в некотором состоянии $q_i \in Q$, считывает очередной символ $w \in \Sigma$ из входной цепочки символов и изменяет свое состояние на $q_{i+1} = \delta(q_i, w)$, после чего указатель в цепочке входных символов передвигается на следующий символ и начинается такт $i+1$. Так продолжается до тех пор, пока цепочка входных символов не закончится. Конец цепочки символов часто помечают особым символом $\perp$. Считается также, что перед тактом 1 автомат находится в начальном состоянии q_0 .

Говорят, что автомат допускает цепочку $\alpha \in \Sigma^*$, если в результате выполнения всех тактов работы над этой цепочкой он окажется в состоянии $q \in F$. Язык, определяемый автоматом, является множеством всех цепочек, допускаемых автоматом. Для анализа цепочки с помощью конечного автомата достаточно подать ее на вход автомата, выполнить все такты его работы и определить, перешел ли автомат в результате работы в одно из заданных конечных состояний.

Графически автомат отображается нагруженным однонаправленным графом, в котором вершины представляют состояния, дуги отображают переходы из одного состояния в другое, а символы нагрузки (пометки) дуг соответствуют функции перехода. Если функция перехода предусматривает переход из состояния q в q' по нескольким символам, то между ними строится одна дуга, которая помечается всеми символами, по которым происходит переход из q в q' .

Недетерминированный автомат неудобен для анализа цепочек, так как в нем могут встречаться состояния, допускающие неоднозначность, т.е. такие, из которых выходит две или более дуги, помеченных одним и тем же символом. Очевидно, что программирование работы такого автомата – нетривиальная задача.

Доказано, что любой недетерминированный автомат может быть преобразован в детерминированный так, чтобы их языки совпадали (говорят, что автоматы эквивалентны).

Детерминированный конечный автомат задается пятеркой:

$$M' = (Q', \Sigma, \delta', q_0', F'),$$

где:

Q' – конечное множество состояний автомата;

Σ – конечное множество допустимых входных символов автомата;

$q_0' \in Q'$ – начальное состояние автомата;

δ' – заданное отображение множества $Q'^* \Sigma$ во множество Q' δ :
 $Q'^* \Sigma \rightarrow Q'$;

$F \subseteq Q'$ – множество заключительных состояний автомата.

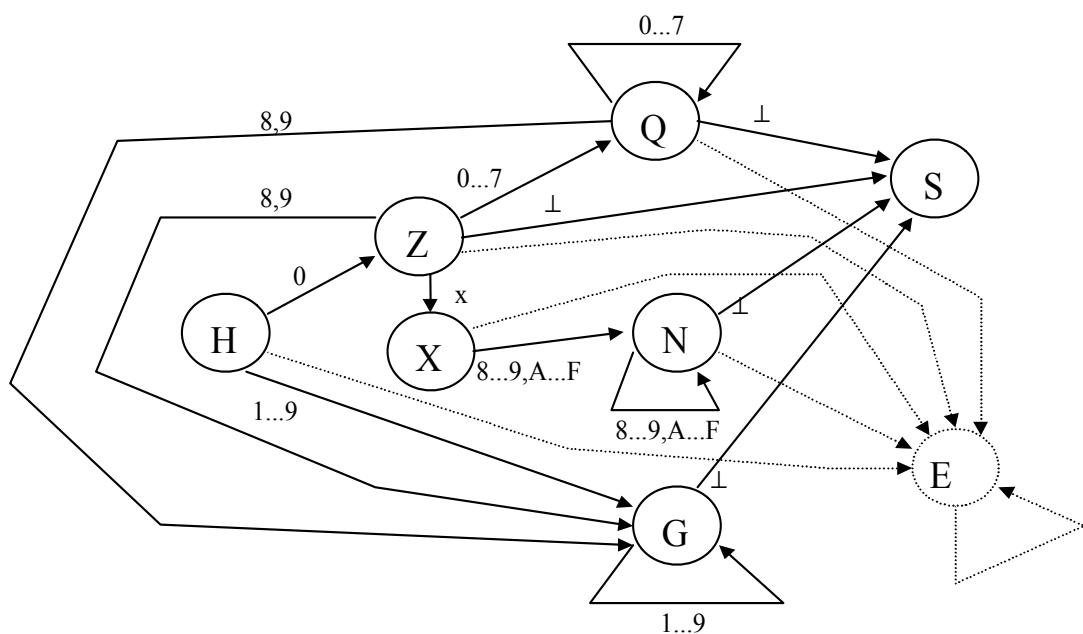
После построения конечный детерминированный автомат может быть минимизирован, т.е. для него может быть построен эквивалентный ему автомат с минимальным числом состояний.

Можно написать функцию, отражающую функционирование любого детерминированного конечного автомата. Чтобы запрограммировать такую функцию, достаточно иметь переменную, которая бы отображала текущее состояние автомата, а переходы автомата из одного состояния в другое на основе символов входной цепочки могут быть построены с помощью операторов выбора. Работа функции должна продолжаться до тех пор, пока не будет достигнут конец входной цепочки. Для вычисления результата функции необходимо по ее завершению проанализировать состояние автомата. Если это одно из конечных состояний, то функция выполнена успешно, и входная цепочка принимается, если нет – то входная цепочка не принадлежит заданному языку.

Рассмотрим пример анализа лексем, представляющих собой целочисленные константы без знака в формате языка Си. В соответствии с требованиями языка, такие константы могут быть десятичными, восьмеричными, либо шестнадцатеричными. Восьмеричной константой считается число, начинающееся с 0 и содержащее цифры от 0 до 7; шестнадцатеричная константа должна начинаться с последовательности символов 0х и может содержать цифры и буквы от А до F (будем рассматривать только прописные буквы). Остальные числа считаются десятичными (правила их записи напоминать, наверное, не стоит). Будем считать, что всякое число завершается символом конца строки $\perp$.

Рассмотренные выше правила могут быть записаны в форме Бэкуса-Наура в грамматике целочисленных констант без знака языка Си (для избегания путаницы терминальные символы грамматики выделены жирным шрифтом).

Рисунок 1.1. Граф конечного детерминированного автомата, распознающего грамматику целых чисел языка Си.



$G(\{0...9, A...F, \perp\}, \{S, G, X, N, Q, Z\}, P, S)$

P: $S \rightarrow G\perp | Z\perp | N\perp | Q\perp$

$G \rightarrow 1|2|3|4|5|6|7|8|9|G0|G1|G2|G3|G4|G5|G6|G7|G8|G9|Z8|Z9|Q8|Q9$

$H \rightarrow X0|X1|X2|X3|X4|X5|X6|X7|X8|X9|XA|XB|XC|XD|XE|XF|$

$N0|N1|N2|N3|N4|N5|N6|N7|N8|N9|NA|NB|NC|ND|NE|NF$

$X \rightarrow Zx$

$Q \rightarrow Z0|Z1|Z2|Z3|Z4|Z5|Z6|Z7|Q0|Q1|Q2|Q3|Q4|Q5|Q6|Q7$

$Z \rightarrow 0$

Хорошо видно, что данная грамматика является регулярной грамматикой (леволинейной). Конечный детерминированный автомат $M'(\{N, Z, X, N, Q, G, S, ER\}, \{0...9, A...F, \perp\}, \delta, H, \{S\})$, который распознает язык, заданный этой грамматикой, изображен на рисунке 1.1. Начальным состоянием автомата является состояние H . В автомат дополнительно введено особое состояние E , обозначающее ошибку в распознавании цепочки символов. На графе автомата дуги, идущие в это состояние, не нагружены символами. По принятому соглашению они обозначают функцию перехода по любому символу, кроме тех символов, которыми уже помечены другие дуги, выходящие из той же вершины графа. Такое соглашение принято, чтобы не загромождать обозначениями граф автомата (на рисунке 1.1 такие дуги и состояние E выделены пунктиром).

Можно написать программу, моделирующую работу указанного автомата. Ниже приводится текст функции на языке Паскаль, его реализующей. Результат функции истинный (*True*), если входная цепочка символов принадлежит входному языку автомата. Границей цепочки считается символ с кодом 0 (*#0*), в функции он искусственно добавляется в конец цепочки.

В программе переменная *iState* отображает текущее состояние автомата, переменная *i* является счетчиком символов входной строки. Конечно, рассмотренная программа может быть оптимизирована (например, можно сразу же прекращать разбор по обнаружению ошибки), но в данном примере оптимизация не выполнялась, чтобы можно было четко отследить соответствие между программой и построенным автоматом.

```
type
  TAutoState = ( AUTO_H, AUTO_Z, AUTO_X,
                  AUTO_Q, AUTO_N, AUTO_G,
                  AUTO_ER, AUTO_S );

function RunAuto (sInput: string): Boolean;
var
  iState : TAutoState;
  i : integer;
begin
  sInput := sInput + #0;
  iState := AUTO_H;
```

```

i := 0;
repeat
    i := i + 1;
    case iState of
        AUTO_H:
            case sInput[i] of
                '0': iState := AUTO_Z;
                '1'..'9': iState := AUTO_G;
                else iState := AUTO_E;
            end;
        AUTO_Z:
            case sInput[i] of
                '0'..'7': iState := AUTO_Q;
                '8','9': iState := AUTO_G;
                'x': iState := AUTO_X;
                #0: iState := AUTO_S;
                else iState := AUTO_E;
            end;
        AUTO_X:
            case sInput[i] of
                '0'..'9': iState := AUTO_N;
                'A'..'F': iState := AUTO_N;
                else iState := AUTO_E;
            end;
        AUTO_Q:
            case sInput[i] of
                '0'..'7': iState := AUTO_Q;
                '8','9': iState := AUTO_G;
                #0: iState := AUTO_S;
                else iState := AUTO_E;
            end;
        AUTO_N:
            case sInput[i] of
                '0'..'9': iState := AUTO_N;
                'A'..'F': iState := AUTO_N;
                #0: iState := AUTO_S;
                else iState := AUTO_E;
            end;
        AUTO_G:
            case sInput[i] of
                '0'..'9': iState := AUTO_G;
                #0: iState := AUTO_S;
                else iState := AUTO_E;
            end;
        AUTO_E: iState := AUTO_E;
    end {case};
until (sInput[i] = #0);
RunAuto := (iState = AUTO_S);
end; { RunAuto }

```

Однако в общем случае задача сканера несколько шире, чем просто проверка цепочки символов лексемы на соответствие ее входному языку. Сканер должен выполнить те или иные действия по запоминанию распознанной лексемы (занесение ее в таблицу лексем). Набор действий определяется реализацией компилятора. Обычно эти действия выполняются сразу же по обнаружению конца распознаваемой лексемы, поэтому их

несложно вставить в соответствующие места рассмотренной выше программы-сканера (в те операторы, где обнаруживается символ #0).

Вторая проблема, которая уже обсуждалась выше, это выделение границ лексем. Ведь во входном тексте лексемы не ограничены специальными символами. Если говорить в терминах программы-сканера, то определение границ лексем – это выделение тех строк в общем потоке входных символов, для которых надо выполнять распознавание. В общем случае эта задача может быть сложной, но для простейших входных языков границы лексем распознаются по заданным терминальным символам. Эти символы – пробелы, знаки операций, символы комментариев, а также разделители (запятые, точки с запятой и др.). Набор таких терминальных символов может варьироваться в зависимости от входного языка. Важно отметить, что знаки операций сами также являются лексемами, и необходимо не пропустить их при распознавании текста.

Таким образом, алгоритм работы простейшего сканера можно описать так:

- просматривается входной поток символов программы на исходном языке до обнаружения очередного символа, ограничивающего лексему;
- для выбранной части входного потока выполняется функция распознавания лексемы;
- при успешном распознавании информация о выделенной лексеме заносится в таблицу лексем, и алгоритм возвращается к первому этапу;
- при неуспешном распознавании выдается сообщение об ошибке, а дальнейшие действия зависят от реализации сканера – либо его выполнение прекращается, либо делается попытка распознать следующую лексему (идет возврат к первому этапу алгоритма).

Работа программы-сканера продолжается до тех пор, пока не будут просмотрены все символы программы на исходном языке из входного потока.

Лексический анализ всегда предшествует синтаксическому анализу и заключается в предварительной обработке программы и ее перекодировании

к виду, удобному для синтаксического анализа. Лексический анализ принято отделять от синтаксического по двум причинам:

- для сокращения времени компиляции;
- синтаксис лексем всегда проще, чем синтаксис программ.

Лексемой называется некоторая конструкция, выступающая как терминальный символ для синтаксического анализа. Лексемы еще иногда называют символами или атомами. Поскольку при генерации машинных команд нужны конкретные значения идентификаторов отдельных лексем, то можно дать следующее определение лексемы.

Лексема – это пара (класс, значение). Значение может быть непосредственным или представлять собой ссылку на некоторую таблицу, в которой хранятся значения лексем. Большинство лексем в языках программирования можно сгруппировать в следующие классы:

- идентификаторы;
- служебные слова;
- числа;
- однолитерные разделители;
- двулитерные разделители.

Классы лексем могут быть описаны простыми правилами, которые либо являются автоматными, либо могут быть приведены к автоматным. Например:

```
<идентиф> ::= буква | <идентиф>буква | <идентиф>цифра  
<число> ::= <целое> | .<целое> | <целое>. | <целое>.<целое>  
<целое> ::= цифра | <целое>цифра  
<разделитель> ::= / | // | /* | . | ;
```

Во внутреннем представлении все лексемы имеют вид пары, в которой первый знак – это признак принадлежности к классу (буква или цифра), а второй – значение в описанном выше понимании.

Рассмотрим пример построения лексического анализатора для языка, являющегося некоторым подмножеством языка ПЛ/1. Определим входной язык.

1. Описание данных: ограничимся только одним типом числовых данных DEC FIXED, то есть вещественное число с фиксированной точкой:

2. Описание процедур;

3. Операторы :

– присваивания = ;

– безусловного перехода GOTO;;

– условный логический оператор IF THEN;

– оператор конца процедуры END.

4. Выражения:

– операции сложения + ;

– умножения * ;

– отношений <, > ;

– скобки (,) .

5. Метки вида <идентиф>: ;

6. Разделители: , ; .

Построим таблицу классов лексем (таблица 2.1).

Таблица 2.1 – Таблица классов лексем

Типы лексем	Обозначение
Служебные слова	W
Идентификаторы	I
Операторы	O
Разделители	R
Числа	N

Лексический анализатор (сканер) должен выделять лексемы из исходной программы и передавать их синтаксическому анализатору во внутреннем представлении. При этом используются таблицы, соответствующие каждому классу лексем. Таблицы служебных слов, разделителей, операций определяются входным языком и должны быть

сформированы при построении сканера, а таблицы идентификаторов и чисел заполняются в процессе разбора исходной программы.

Для рассматриваемого языка сформированные таблицы служебных слов (табл. 2.2), таблица операций (табл. 2.3) и таблица разделителей (табл. 2.4) имеют вид:

Таблица 2.2 – Таблица служебных слов

Служебное слово	Код
DEC	1
DCL	2
END	3
FIXED	4
GOTO	5
IF	6
PROG	7
THEM	8
MAIN	9

Таблица 2.3 – Таблица операций

Операция	Код
+	1
*	2
<	3
>	4
=	5
:	6

Таблица 2.4 – Таблица разделителей

Разделитель	Код
Пробел	1
,	2
:	3
(	4
)	5
.	6

Каждая лексема, обработанная сканером, преобразуется к виду:

<буква><код>, где:

<буква> – это признак класса лексемы (W, I, O, R, или N),

<код> – код лексемы в соответствующей таблице.

Например, если на сканер поступила цепочка вида:

```
IF C > 2.79 THEN B = A * E3:
```

то сканер выдаст следующую последовательность:

W6 – служебное слово IF,

I1 – идентификатор C,

04 – операция > ,

N1 – число 2.79,

W8 – служебное слово THEN

I2 – идентификатор B,

05 – операция = ,

I3 – идентификатор A,

02 – оператор * ,

I4 – идентификатор E3,

R3 – разделитель .

Рассмотрим работу сканера на следующем примере:

```
PROC MAIN;
  DCL (A1,A2) DEC FIXED;
  A1=378;
```

```

A2=.73;
PROC CALC;
DCL (SUM,MULT) DEC FIXED;
IF A1+A2>3.2 THEN GOTO P;
SUM=(A1+A2)*A2,
P:  MULT=A1*A2;
END;
END.

```

После работы лексического анализатора программа преобразуется к следующему виду:

```

PROC MAIN;                                W7 W9 R3
      DCL (A1,A2) DEC FIXED;              W2 R4 I1 R2 I2 R5 W1 W4 R3
      A1=378;                             I1 05 N1 R3
      A2=.73;                             I2 05 N2 R3
      PROC CALC;                          W7 I3 R3
DCL SUM. MULT DEC FIXED;                  W2 R4 I4 R2 I5 R5 W1 W4 R3
IF A1+A2>3.2 THEN GOTO P;                 W6 I1 01 I2 04 N3 W8 W5 I6 R3
SUM=A1+A2;                                I4 05 I2 01 I1 R3
P: MULT=A1*(A2+SUM);                      I6 06 I5 05 I1 R4 I2 01 I4 R5 R3
END;                                       W3 R3
END.                                       W3 R6

```

В процессе работы лексического анализатора были сформированы таблица чисел N (табл. 2.5) и таблица идентификаторов I (табл. 2.6) .

Таблица 2.5 – Таблица чисел

Код	Число
1	378
2	.73
3	3.2

Таблица 2.6 – Таблица идентификаторов

Код	Идентификатор	Номер процедуры	Уровень процедуры	Номер идентиф. в процедуре	Тип идентиф.	Объем памяти
1	A1					
2	A2					
3	CALC					
4	SUM					
5	MULT					
6	P					

Нарисуем диаграмму состояний сканера:

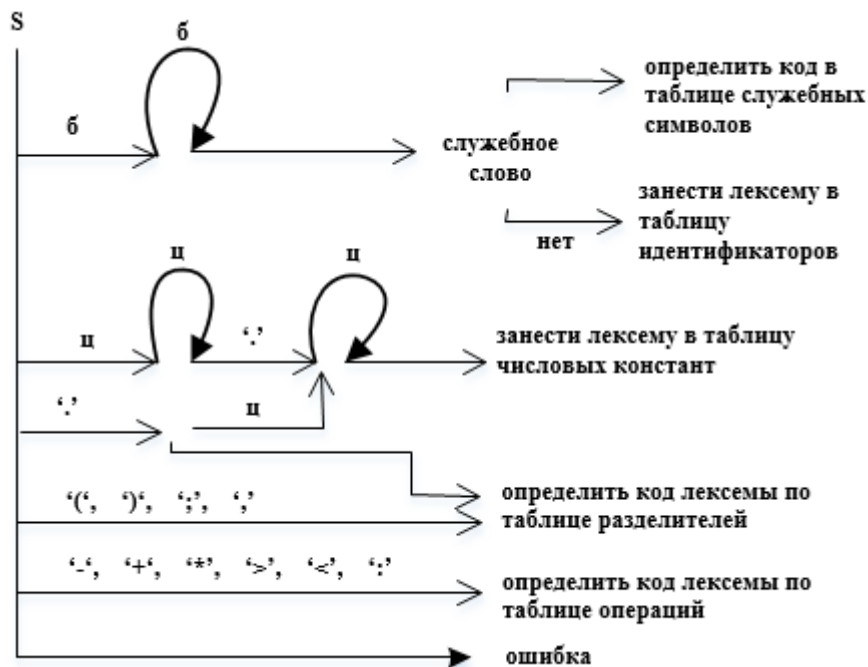


Рисунок 2.1 – Диаграмма состояний сканера

Если в диаграмме состояния есть невзвешенные дуги, то они взвешены любыми символами кроме тех, которыми взвешены другие дуги, исходящие из данного состояния.

3. Аппаратура и материалы

Для выполнения лабораторной работы рекомендуется использовать персональный компьютер со следующими характеристиками: 32-разрядный

(x86) или 64-разрядный (x64) процессор с тактовой частотой 1,5 ГГц и выше, оперативная память – 1 Гб и выше, свободное дисковое пространство – не менее 1 Гб, графическое устройство DirectX 9. Программное обеспечение: операционная система на базе WINDOWS, Microsoft Visual Studio 2008 и старше.

4. Указания по технике безопасности

Техника безопасности при выполнении лабораторной работы совпадает с общепринятой для пользователей персональных компьютеров. Самостоятельно не производить ремонт персонального компьютера, установку и удаление программного обеспечения; в случае неисправности персонального компьютера сообщить об этом обслуживающему персоналу лаборатории (оператору, администратору); соблюдать правила техники безопасности при работе с электрооборудованием; не касаться электрических розеток металлическими предметами; рабочее место пользователя персонального компьютера должно содержаться в чистоте; не разрешается возле персонального компьютера принимать пищу, напитки.

5. Содержание отчета и его форма

1. Номер и название лабораторной работы.
2. Цели лабораторной работы.
3. Ответы на контрольные вопросы.
4. Экранные формы, показывающие порядок выполнения лабораторной работы, и результаты, полученные в ходе её выполнения:

Отчет о выполнении лабораторной работы в письменном виде сдается преподавателю.

6. Задание

1. Разработайте программу, которая выполняет лексематический анализ текста в формате JSON и переводит его в набор лексем. Классы лексем, и их идентификацию продумайте самостоятельно.

2. В качестве исходного файла используйте файл `lr1.json`.
3. Результат преобразования программа должна сохранить в виде текстового файла. Для каждой лексемы должно быть записано:
 - класс лексемы;
 - код лексемы;
 - исходная запись лексемы в тексте.
4. Для разработки программы запрещается использовать какие-либо дополнительные библиотеки или программы (кроме стандартных в используемом вами ЯП).

7. Защита работы

Перед выполнением лабораторной работы каждый студент получает индивидуальное задание.

Защита лабораторной работы происходит только после его выполнения (индивидуального задания). При защите лабораторной работы студент:

- 1) отвечает на контрольные вопросы;
- 2) поясняет ход выполнения индивидуального задания;
- 3) поясняет результаты, полученные в результате выполнения индивидуального задания.

Ход защиты лабораторной работы контролируется преподавателем.